

7-index

Perancangan Basis Data • 24 slides • Generated on 15 April 2026, 08:11

Perancangan Basis Data

Mengapa Performa Query Penting?

Slide 2

Bayangkan tabel dengan 10 juta baris data...

- **Tanpa indeks: MySQL harus memeriksa setiap baris satu per satu (Full Table Scan)**
- Query sederhana seperti WHERE email = 'john@mail.com' bisa memakan waktu detik hingga menit
- Performa makin buruk seiring bertambahnya data (kompleksitas $O(n)$)
- Beban CPU & I/O disk meningkat drastis pada operasi JOIN, ORDER BY, GROUP BY SELECT * FROM users WHERE email = 'john@mail.com'; -- 10 juta baris diperiksa, hanya 1 yang cocok!

Query ke tabel dengan dan tanpa indeks Perhatikan perbedaan waktu eksekusi

Definisi

- Indeks adalah struktur data tambahan yang dibuat di atas tabel untuk mempercepat pencarian data
- MySQL InnoDB menggunakan struktur B+Tree sebagai implementasi default
- Mengubah pencarian dari $O(n)$ menjadi $O(\log n)$ Analogi: Daftar Isi Buku
- Tanpa indeks = membaca buku 1.000 halaman dari awal untuk mencari satu kata
- Dengan indeks = melihat daftar isi, langsung ke halaman yang tepat Indeks digunakan oleh: WHERE, JOIN, ORDER BY, GROUP BY

Cara Kerja B+Tree Index

[30 | 60]

[10 | 20]

[40 | 50]

[70 | 80]

5|8|10

15|18|20

35|38|40

45|48|50

65|70|75

80|85|90

Root

Internal

Leaf

→ Leaf nodes saling terhubung (linked list) untuk range scan

Karakteristik B+Tree pada InnoDB:

- Semua data tersimpan di leaf node; internal node hanya menyimpan key
- Setiap node = 1 page InnoDB (default 16KB)
- Pencarian: Root → Internal → Leaf = $O(\log n)$
- 10 juta baris hanya butuh ~3-4 level lookup

Clustered vs Secondary Index

Slide 6

Clustered Index

- Data fisik tersusun sesuai urutan indeks
- **Hanya 1 per tabel (= PRIMARY KEY)**
- **Leaf node menyimpan seluruh data baris**
- Pencarian by PK sangat cepat `SELECT * FROM users WHERE id = 5;` -- langsung ke leaf!

Secondary Index

- Indeks terpisah dari data fisik
- **Bisa banyak per tabel**
- **Leaf node menyimpan key + pointer ke PK**
- **Perlu bookmark lookup ke clustered index** `SELECT * FROM users WHERE email = 'a@b.com';` -- idx_email -> PK lookup

- **PRIMARY KEY** — Clustered index, satu per tabel, data tersusun fisik
- **UNIQUE INDEX** — Menjamin tidak ada duplikasi pada kolom tertentu
- **INDEX (Standard)** — Secondary index untuk mempercepat pencarian
- **COMPOSITE INDEX** — Indeks pada beberapa kolom sekaligus
- **FULLTEXT INDEX** — Pencarian teks penuh pada VARCHAR/TEXT
- **SPATIAL INDEX** — Untuk data geografis (**GEOMETRY**) Semua jenis indeks di atas (kecuali Fulltext & Spatial) menggunakan B+Tree

Primary Key (Clustered Index)

Slide 8

- **Dibuat otomatis saat mendefinisikan PRIMARY KEY**
- **Data disimpan secara fisik terurut berdasarkan primary key**
- **Hanya satu clustered index per tabel**
- Jika tidak ada PK, InnoDB membuat hidden clustered index
`CREATE TABLE users ( id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), email VARCHAR(100) );`

Unique Index

Slide 9

- **Menjamin tidak ada nilai duplikat pada kolom yang diindeks**
- **Membolehkan NULL (berbeda dengan PRIMARY KEY yang NOT NULL)**
- Bisa lebih dari satu per tabel
- Cocok untuk: email, username, no_ktp, no_telepon
`CREATE UNIQUE INDEX idx_email ON users(email);` -- INSERT duplikat email akan ERROR!

Indeks Standar (Secondary Index)

Slide 10

- **Indeks paling umum digunakan (non-unique secondary index)**
- Tidak mengubah penyimpanan fisik data
- Membolehkan nilai duplikat
- **Cocok untuk kolom yang sering ada di WHERE, JOIN, ORDER BY** `CREATE INDEX idx_last_name ON users(last_name);` -- Juga bisa saat CREATE TABLE: `CREATE TABLE users ( ..., INDEX idx_name (last_name) );`

Composite Index & Leftmost Prefix Rule

Slide 11

- Indeks pada beberapa kolom sekaligus: `INDEX(a, b, c)`
- Mengikuti Leftmost Prefix Rule — indeks hanya digunakan dari kolom paling kiri `CREATE INDEX idx_abc ON t(a, b, c);`
`WHERE a = 1 AND b = 2 AND c = 3` ✓ Pakai penuh `WHERE a = 1 AND b = 2` ✓ Pakai a,b `WHERE a = 1` ✓ Pakai a `WHERE b = 2 AND c = 3` ✗ Tidak terpakai! `WHERE c = 3` ✗ Tidak terpakai!

- **Untuk pencarian teks pada kolom VARCHAR atau TEXT**
- Jauh lebih efisien dari LIKE '%keyword%'
- **Mendukung pencarian berbasis relevansi (natural language mode)** `CREATE FULLTEXT INDEX idx_desc ON products(description); SELECT * FROM products WHERE MATCH(description) AGAINST ('laptop gaming');`

Membuat & Mengelola Indeks

Slide 13

Membuat indeks: CREATE INDEX idx_name ON tabel(kolom); CREATE UNIQUE INDEX idx ON tabel(kolom); **Melihat indeks yang ada:** SHOW INDEX FROM tabel; **Menghapus indeks:** DROP INDEX idx_name ON tabel; **Menganalisis query:** EXPLAIN SELECT * FROM tabel WHERE kolom = 'x';

Kolom penting pada output EXPLAIN:

- **type** — Metode akses (dari terbaik ke terburuk):
 - **const** > **ref** > **range** > **index** > **ALL**
- **key** — Indeks yang benar-benar digunakan (**NULL** = tidak ada)
- **rows** — Estimasi jumlah baris yang diperiksa (makin kecil makin baik)
- **Extra** — Info tambahan:
 - Using index = covering index (sangat optimal)
 - Using filesort = sorting tanpa indeks (lambat)
 - Using temporary = perlu tabel sementara (lambat)

Praktik EXPLAIN dengan dan tanpa indeks Amati kolom: type, key, rows, Extra

Best Practice: Pemilihan Kolom untuk Index

Slide 16

- **High Cardinality — Pilih kolom dengan banyak nilai unik**
 - ✓ email, username, no_ktp (jutaan nilai unik)
 - ✗ gender, status, is_active (2-5 nilai saja)
- **Kolom yang sering di-query — WHERE, JOIN, ORDER BY, GROUP BY**
- **Covering Index — Indeks yang memuat semua kolom yang dibutuhkan query**
 - Tidak perlu lookup ke tabel utama (Using index di EXPLAIN)
- **Foreign Key — Selalu index kolom FK untuk performa JOIN**

Aturan Urutan Kolom: Equality → Range → Sort

- Kolom dengan kondisi = (equality) diletakkan paling kiri
- Kolom dengan kondisi range (>, <, BETWEEN) di tengah
- Kolom ORDER BY di paling kanan Contoh: SELECT * FROM orders WHERE status = 'shipped' ← equality AND created_at > '2024-01-01' ← range ORDER BY total_amount DESC; ← sort **CREATE INDEX idx_orders ON orders(status, created_at, total_amount);**

Kesalahan Umum #1: Fungsi pada Kolom WHERE

Slide 18

✗ **SALAH** — Fungsi pada kolom membatalkan penggunaan indeks! WHERE DATE(created_at) = '2024-06-15' WHERE YEAR(created_at) = 2024 WHERE LOWER(email) = 'john@mail.com' WHERE price + 10 > 100 ✓ **BENAR** — Tulis ulang agar kolom tetap "bersih": WHERE created_at >= '2024-06-15' AND created_at < '2024-06-16' WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31' WHERE price > 90

Kesalahan Umum #2: Pattern & Konversi Implisit

Slide 19

X LIKE dengan wildcard di depan: WHERE name LIKE '%smith%' ← full table scan! WHERE name LIKE 'smith%' ← bisa pakai indeks **X**

Konversi tipe data implisit: WHERE phone = 08123456789 ← phone VARCHAR, dibanding INT WHERE phone = '08123456789' ← cocok, index dipakai **X**

OR tanpa indeks pada semua kolom: WHERE name = 'John' OR city = 'Jakarta' **Solusi: buat indeks pada kedua kolom**

Kesalahan Umum #3: Over-Indexing

Slide 20

- **Terlalu banyak indeks (over-indexing)**
- **Setiap INSERT/UPDATE/DELETE harus memperbarui semua indeks**
- Memakan ruang disk tambahan (bisa 2-3x ukuran tabel)
- Optimizer MySQL bisa bingung memilih indeks
- **Indeks yang tidak pernah dipakai**
- Cek dengan: `sys.schema_unused_indexes`
- **Indeks duplikat / redundan**
- `INDEX(a) + INDEX(a, b) → INDEX(a)` redundan!
- Cukup `INDEX(a, b)` saja (leftmost prefix rule!)

Tabel: orders (10 juta baris) id | user_id | status | total | created_at | shipping_city **Query yang sering digunakan:**

- WHERE user_id = ? ORDER BY created_at DESC
- WHERE status = ? AND created_at BETWEEN ? AND ?
- WHERE shipping_city = ? AND status = ? ✓ **Desain indeks yang optimal:** CREATE INDEX idx_user_date ON orders(user_id, created_at); CREATE INDEX idx_status_date ON orders(status, created_at); CREATE INDEX idx_city_status ON orders(shipping_city, status); Prinsip: 1 indeks per pola query utama, equality kolom di kiri

Tabel: mahasiswa, nilai, mata_kuliah mahasiswa: id | nim | nama | jurusan | angkatan nilai: id | mhs_id | mk_id | semester | skor

Analisis kebutuhan index:

- Cari mahasiswa by NIM: UNIQUE INDEX(nim)
- Filter jurusan + angkatan: INDEX(jurusan, angkatan)
- FK pada tabel nilai: INDEX(mhs_id), INDEX(mk_id)
- KHS per semester: INDEX(mhs_id, semester) **Pertanyaan: Apakah INDEX(mhs_id) masih diperlukan jika sudah ada INDEX(mhs_id, semester)?** Jawab: TIDAK — leftmost prefix rule!

Latihan: Pilih Index yang Tepat

Slide 23

Tabel: products (id, name, category, price, stock, created_at) **Soal 1: Indeks apa untuk query ini?** `SELECT * FROM products WHERE category = 'Laptop' AND price < 10000000 ORDER BY created_at;` **Soal 2: Mengapa query ini tidak menggunakan idx_price?** `SELECT * FROM products WHERE price + 100 > 500000;` **Soal 3: Mana yang lebih baik dan mengapa?** a) `INDEX(stock) + INDEX(category)` b) `INDEX(category, stock)` -- untuk: `WHERE category = ? AND stock > 0`

- Indeks adalah struktur data B+Tree yang mempercepat pencarian $O(n) \rightarrow O(\log n)$
- Jenis utama: Primary, Unique, Standard, Composite, Fulltext
- Composite index mengikuti leftmost prefix rule
- Urutan kolom composite: Equality $\rightarrow$ Range $\rightarrow$ Sort
- Hindari: fungsi pada WHERE, LIKE '%...', over-indexing
- Selalu gunakan EXPLAIN untuk memverifikasi indeks terpakai